

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»

ОТЧЕТ
О ПРОЕКТНОЙ РАБОТЕ

РАЗРАБОТКА TELEGRAM-ЧАТ-БОТА С ФУНКЦИЕЙ РАСПОЗНАВАНИЯ ЭМОЦИЙ
СРЕДСТВАМИ МАШИННОГО ОБУЧЕНИЯ

Выполнили:
Кремпольская Екатерина
Александровна

Касьяненко Вера
Михайловна

Студенты группы:
Р3320

Преподаватель:
Русак Алена Викторовна

ОГЛАВЛЕНИЕ

1. Тема проекта	4
1.1 Аннотация	4
1.2 Цель	4
1.3 Актуальность	4
2. Анализ предметной области	5
2.1 Определение эмоций по тексту	5
2.2 Определение эмоций по голосу	6
2.3 Паттерны эмпатичных ответов	7
2.4 Психологические факторы, влияющие на коммуникацию	8
3. Определение задач работы	10
3.1 Задачи	10
3.2 Задачи Веры Касьяненко	10
3.3 Задачи Екатерины Кремпольской	10
4. Экспериментальная часть: проверка Dense, LSTM, GRU и CNN	11
4.1 Архитектура Dense-модели	11
4.1.1 Настройка и обучение	11
4.1.2 Результаты обучения	11
4.2 Архитектура LSTM-модели	13
4.2.1 Настройка и обучение	13
4.2.2 Результаты обучения	14
4.3 Архитектура GRU-модели	15
4.3.1 Настройка и обучение	15
4.3.2 Результаты обучения	16
4.4 Архитектура CNN-модели	17
4.4.1 Настройка и обучение	18
4.4.2 Результаты обучения	18
4.5 Вывод	19
5. Проектирование и разработка. Работа с текстом	20
5.1 Библиотеки и модули	20
5.2 Подготовка данных	20
5.2.1 Загрузка набора данных	20
5.2.2 Очистка и предобработка текста	20
5.3 Извлечение признаков	21
5.3.1 Векторизация текста	21
5.4 Построение и обучение модели	21
5.5 Сохранение и упаковка результатов	22
5.6 Оценка модели	22
5.7 Визуализация результатов	22
6. Проектирование и разработка. Работа с аудио	25
6.1 Библиотеки и модули	25
6.2 Загрузка датасетов	25

6.2.1 Визуализация распределения эмоций	26
6.2.2 Предварительный просмотр и предобработка аудио	26
6.3 Извлечение признаков	27
6.3.1 Подготовка обучающих данных	28
6.4 Построение и обучение модели	28
6.5 Визуализация динамики обучения	30
7. Реализация Telegram-бота	33
7.1 Подробное описание	33
7.2 Взаимодействие компонентов	33
8. Исследование продукта	35
9. Апробация Telegram-бота на реальных пользователях	37
10. Выводы по выполненной работе	40
Список использованных источников	41
Приложение А Ссылки на исходные коды	42

1. ТЕМА ПРОЕКТА

Telegram-чат-бота, который способен распознавать эмоцию пользователя по одному входящему сообщению (текстовому или голосовому) и выдавать одноразовое поддерживающее сообщение в ответ.

1.1 Аннотация

Чат-боты все чаще используются для эмоциональной поддержки и сопровождения пользователей в цифровой среде. Данный проект посвящен разработке Telegram-чат-бота, который способен распознавать эмоцию пользователя по одному входящему сообщению (текстовому или голосовому) и выдавать поддерживающее сообщение в ответ. Такой формат особенно полезен для быстрого эмоционального облегчения – пользователь получает сразу же поддержку. В рамках проекта реализованы модули анализа текста и речи для определения эмоционального состояния пользователя, а также механизм отправки поддерживающих высказываний по выявленной эмоции. При формировании ответов упор сделан на паттерны поддерживающих высказываний, описанные в лингвистических исследованиях, с целью сделать ответы бота максимально эмпатичными.

1.2 Цель

Автоматизация определения эмоционального состояния пользователя по текстовым или голосовым сообщениям и выработка рекомендаций по регулированию эмоционального состояния.

1.3 Актуальность

Современное общество сталкивается с возрастающим уровнем стресса, эмоционального напряжения. На этом фоне всё большую значимость приобретают технологии, способные обеспечивать оперативную эмоциональную поддержку. Одним из перспективных направлений является применение интеллектуальных чат-ботов, способных распознавать эмоциональное состояние пользователя и реагировать соответствующим образом.

Мессенджеры, такие как Telegram, стали повседневным каналом общения для миллионов пользователей, что делает их удобной средой для внедрения подобных решений. А применение нейросетевых моделей позволяют достаточно точно определять эмоции по коротким сообщениям.

Таким образом, разработка Telegram-бота, определяющего эмоции пользователя по одному сообщению и предоставляющего эмоционально релевантную поддержку, представляет собой актуальную задачу.

2. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 ОПРЕДЕЛЕНИЕ ЭМОЦИЙ ПО ТЕКСТУ

Изучив статью [1], мы выяснили, что распознавание эмоционального состояния по текстовым данным активно исследуется и базируется на нескольких подходах. Ранние методы полагались на лексиконный анализ: наличие определённых “эмоциональных” ключевых слов или выражений в тексте.

Такой ключевой подход с использованием словарей (например, WordNet Affect, SentiWordNet) и правил позволяет сопоставить словам метки эмоций. Он требует предварительной обработки текста – токенизации, удаления стоп-слов, лемматизации.

Однако чисто словарный метод ограничен: если в сообщении отсутствуют явные эмоциональные слова, эмоция может остаться нераспознанной.

Дальнейшее развитие получили методы машинного обучения. Здесь эмоциональная окраска текста определяется как задача многоклассовой классификации на основе признаков текста. Используются обучающие корпуса размеченных данных и алгоритмы как Naïve Bayes, SVM и другие для классификации высказывания по эмоциям.

Такие модели способны учитывать статистические соотношения слов и эмоций и достигали точности порядка 70–90% на отдельных наборах данных.

Например, в одном исследовании комбинация SVM, Naïve Bayes и дерева решений на потоках текстовых сообщений позволила достичь около 90% точности определения эмоции.

Тем не менее, классические алгоритмы ML часто не учитывают контекстной последовательности слов. В последние годы доминируют нейросетевые методы глубокого обучения для анализа текста. Рекуррентные нейронные сети (RNN), особенно архитектуры LSTM и GRU, учитывают порядок слов и долгосрочные зависимости, что критично для понимания подтекста эмоции.

Такие модели обходят проблему «мешка слов» и способны улавливать сложные языковые паттерны. Например, модель на базе двунаправленной GRU, фокусирующаяся на латентных представлениях текста, показала существенный прирост качества по сравнению с традиционными алгоритмами (увеличение F-меры примерно на 26.8 пунктов).

Также внедряются механизмы внимания (attention) и трансформеры. Так, использование предобученной модели BERT (Bidirectional Encoder Representations from Transformers) позволяет получать контекстуальные эмбединги слов и уже дало улучшение метрик в многоязычных экспериментах по эмоциям в тексте.

В совокупности, глубокие нейросети сейчас достигают наивысших результатов. В частности, сравнительный анализ показал, что сеть прямого распространения уступает рекуррентной: лучшая модель на основе нейросети успешно распознала 6 эмоций в тексте, продемонстрировав, например, F1-меру 0.95 для класса «печаль».

Эти высокие показатели значительно превосходят точность лексиконного и классического ML подходов. Таким образом, использование глубокого обучения (включая RNN и трансформеры) является наиболее эффективным методом определения эмоций по тексту на сегодняшний день.

Обзор современных методик показывает трёхступенчатую эволюцию: лексикон-правил-базированные методы, классический ML, глубокое обучение. Первая методика опирается на словари вроде WordNet Affect или SentiWordNet и дают быстрое, но поверхностное обнаружение эмоций. Большое мета-исследование [2] подчёркивает, что при отсутствии явных эмоциональных ключевых слов такие системы пропускают до 30 % примеров. Алгоритмы NB и SVM, обученные на размеченных корпусах, уже учитывают статистику со-встречаемости слов; в работе 2023 г. [1] **шесть эмоций на датасете 20 000 постов были классифицированы с macro-F1 $\approx$ 0,88, причём модель SVM обеспечила 0,82, а NB — 0,79. RNN-архитектуры устраняют проблему «мешка слов». В эксперименте [3] с Bi-GRU переход к латентным представлениям дал прирост F-меры на 26,8 пунктов относительно SVM + BoW. В той же категории трансформер-энкодеры (BERT-варианты) повышают точность без спец-фич, благодаря контекстуальным эмбедингам.**

Итого, мы можем подытожить, что гибрид BERT-fine-tuning + тонкая дообучаемая GRU-голова сейчас даёт лучшее соотношение качество/затраты и поэтому выбран в нашем проекте.

2.2 ОПРЕДЕЛЕНИЕ ЭМОЦИЙ ПО ГОЛОСУ

Обратившись к статье [4], мы выяснили, что эмоциональная окраска речи выявляется по характеристикам аудиосигнала. Акустические признаки голоса – тон (частота основного тона), громкость, тембр, а также спектральные параметры – существенно меняются в зависимости от эмоционального состояния говорящего. Для количественного описания этих свойств широко применяются Mel-frequency cepstral coefficients (MFCC) – коэффициенты мел-частотного кепстра, дающие сжатое представление спектра речи.

В традиционных подходах признаки извлекаются из аудио и подаются в классические модели классификации (SVM, Random Forest) для определения эмоции.

Современные методы используют глубокие нейронные сети для анализа звука. Применяются сверточные нейросети (CNN), которые обучаются на спектрограммах или мел-спектрограммах речи и выявляют характерные локальные частотные паттерны, соответствующие эмоциям (например, резкие высокочастотные всплески для гнева, монотонность для грусти). Также используются рекуррентные сети (LSTM/GRU) для учета временной динамики эмоциональных интонаций. Нередко архитектуры комбинируют эти подходы: сверточные слои для выделения признаков и bidirectional LSTM для учета последовательности изменения спектра во времени. В реализованной системе выбран именно такой гибридный подход: модель включает подряд несколько Conv1D-слоев с максимальным пуллингом, извлекающих особенности из аудиопотока, и двухслойную BiLSTM, обрабатывающую последовательность признаков, за которой следуют плотные слои классификатора. Данная модель классифицирует речь на несколько категорий эмоций по голосу (в нашем случае – 5 классов эмоций, объединённых из исходных, для устойчивости обучения).

Акустические признаки (F_0 , энергия, спектральная огибающая, MFCC) напрямую связаны с эмоциональной активацией. Пост-обзор 2020 г. [4] предлагает Deep Dual Recurrent Encoder: Conv1D-блоки выделяют частотные паттерны, затем Bi-LSTM анализирует временную динамику; параллельно аналогичная ветка обрабатывает транскрибированный текст. Такая мультимодальная схема увеличивает macro-F1 детекции на ≈ 9 пунктов по сравнению с чисто аудио-CNN.

Отдельно стоит отметить подходы, объединяющие аудио- и текстовую модальность. Поскольку в речи содержится не только звуковой сигнал, но и словесный контент, возможно извлечение эмоций как напрямую из голоса, так и из распознанного текста речи. Также в данной работе была предложена архитектура Deep Dual Recurrent Encoder, в которой одновременно обрабатываются акустические признаки речи и транскрибированный текст высказывания. Идея состоит в том, чтобы анализировать сигнал на «низком уровне» (интонации, тембр) и параллельно на «языковом уровне» (содержание сказанного). Такой подход позволяет компенсировать ограничения каждого из каналов: например, алгоритм учтёт как эмоциональную манеру произнесения, так и наличие эмоционально окрашенных слов. Это особенно важно, учитывая проблемы распознавания эмоций только по звуку: ограниченное количество размеченных данных речи затрудняет обучение сложных моделей, разные люди выражают эмоции по-разному, и некоторые тонкие отличия сложно уловить только на основании аудиосигнала.

Объединение с информацией из текста повышает надёжность детекции. Таким образом, в области Speech Emotion Recognition наилучшие результаты достигаются либо глубокими архитектурами, обученными на больших данных, либо мульти-модальным анализом речи и текста одновременно.

2.3 ПАТТЕРНЫ ЭМПАТИЧНЫХ ОТВЕТОВ

Целью проекта является не только классификация эмоции пользователя, но и предоставление сочувственного отклика. Для этого необходимо определить, какими характеристиками обладает эмпатичный (поддерживающий) ответ. В психологии коммуникации проработаны критерии качественной поддержки. В частности, как отмечают исследователи в области межличностной коммуникации, эффективное поддерживающее сообщение должно валидировать эмоции собеседника (признать и принять его чувства) и контекстуализировать ситуацию, то есть показать понимание причин переживания [5]. Также полезно помочь адресату проанализировать проблему и при необходимости мягко предложить способ справиться.

Проявление эмпатии часто выражается через перефразирование переживаний (“Я понимаю, что ты чувствуешь...”), подтверждение значимости эмоций (“Твоя боль понятна и естественна”) и предложение поддержки (“Я рядом, мы справимся”) – такие компоненты повышают воспринимаемую искренность и полезность ответа. Психолингвистические исследования показывают, что выработка подобных ответов – непростая задача даже для людей.

Исследование 209 участников в соцсетях [5] показало: при ответе на негативный пост люди ощущают меньшую «эф-эффективность» (self-efficacy) и дают короче, менее позитивные отклики. Чтобы смягчить это, эффективный поддерживающий ответ должен:

- валидировать чувство («Я вижу, как тебе тяжело...»),
- демонстрировать понимание контекста,
- содержать предложение помощи или совместного поиска решения.

Именно эти три блока (валидация → контекст → поддержка) заложены в шаблоны ответов бота.

Более того, в данной работе отмечается, что при ответе на негативное откровение (например, пост о грусти или проблеме) люди склонны тщательнее подбирать слова и испытывать беспокойство о том, как их поддержка будет воспринята.

В социальных сетях замечено, что ответы на негативные сообщения обычно короче и содержат меньше позитивной лексики по сравнению с реакцией на позитивные новости.

Это объясняется тем, что поддерживающий опасается сказать что-то не то и усугубить ситуацию. Для качественной эмпатической реакции важно соблюсти баланс: быть искренне заботливым, тактичным, избегать обесценивания проблемы, но при этом стараться придать сообщению позитивный, обнадеживающий оттенок.

Ключевые паттерны сочувственных высказываний включают слова эмпатии («жаль, что тебе пришлось это пережить»), предложения помощи или поддержки («если тебе нужно, я всегда готов выслушать»), а также валидирующие фразы, показывающие понимание чувств («твои эмоции вполне оправданы в этой ситуации»). Данные принципы легли в основу шаблонов ответов нашего бота.

2.4 ПСИХОЛОГИЧЕСКИЕ ФАКТОРЫ, ВЛИЯЮЩИЕ НА КОММУНИКАЦИЮ

При разработке системы необходимо учитывать, что разные пользователи могут по-разному выражать и воспринимать эмоции. Существуют психологические особенности, которые влияют на эмоциональную коммуникацию, например, алекситимия.

Согласно исследованию [6], у 25 % студентов гуманитарных направлений и лишь у ~10 % технических алекситимия выражена ярко; напротив, 75 % «технарей» показывают норму [6]. Следовательно, часть пользователей будет подавать «плоские» эмоциональные сигналы — модель должна уметь выводить скрытые эмоции из контекста.

Также в другом исследовании [7] говорится, что при впервые диагностированной депрессии ($n = 96$, 2020) низкий эмоциональный интеллект и интроверсия коррелировали с баллом по шкале Гамильтона $r=0,46$ и $r=0,38$ соответственно.

Для нашего проекта это означает, что некоторые пользователи могут предъявлять очень скудные или неоднозначные эмоциональные сигналы в тексте или голосе. Бот должен быть способен работать даже с такими «нев्यраженными» эмоциями – например, распознавать скрытую грусть в нейтральных на первый взгляд сообщениях. Ещё один фактор – наличие депрессивных состояний и связанные с ними личностные особенности. Депрессия широко распространена и сопровождается изменениями как в эмоциональном выражении, так и в восприятии поддержки.

Установлено, что определённые черты личности (низкий эмоциональный интеллект, интровертированность) коррелируют с склонностью к депрессии.

Для системы распознавания это означает, что сообщения от пользователей, находящихся в депрессивном состоянии, могут отличаться лексически (например, преобладанием негативно окрашенных слов, частым использованием местоимения «я» и отрицаний). Кроме того, такие пользователи особенно нуждаются в деликатном, позитивно подкрепляющем ответе, учитывающем их низкий эмоциональный фон. Таким

образом, мы сможем сделать распознавание эмоций более точным, а отклики бота – более персонализированными и эффективными в плане психологической поддержки.

3. ОПРЕДЕЛЕНИЕ ЗАДАЧ РАБОТЫ

3.1 Задачи

- Исследование существующих методов распознавания эмоций в тексте и речи;
- Изучение психологических методик;
- Сбор и анализ датасетов с эмоциональной окраской (текстовых и аудио файлов);
- Подбор архитектур нейронных сетей для каждой области анализа;
- Предобработка текстовых и аудио данных, включая фильтрацию шума и извлечение признаков;
- Обучение модели машинного обучения для анализа текста и речи;
- Оценка точности модели и её корректировка на основе метрик (точность, F1-score и другие);
- Реализация функционала для чат-бота: создание списка рекомендаций для каждой эмоции, прописание возможных реакции на действия пользователя;
- Организация тестирования, а затем апробация Telegram-бота.

3.2 Задачи Веры Касьяненко

- Изучение психологических техник
- Разработка сценариев пользовательского взаимодействия Telegram-бота
- Поиск датасетов с аудио материалами
- Предобработка аудиосигналов, фильтрация шумов
- Обучение модели для анализа аудио
- Оценка качества модели
- Интегрирование модели в чат-бот
- Тестирование бота

3.3 Задачи Екатерины Кремпольской

- Поиск датасетов с текстовыми материалами
- Предобработка датасета
- Обучение модели для анализа текста
- Оценка качества модели
- Интегрирование модели в чат-бот
- Тестирование бота

4. ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ: ПРОВЕРКА DENSE, LSTM, GRU и CNN

Для наилучшего результата, мы построили 4 модели, чтобы выяснить, какая из них лучше всего справляется с задачей.

4.1 АРХИТЕКТУРА DENSE-МОДЕЛИ

```
inputs = Input(shape=(1,), dtype='string')
x = text_vectorizer(inputs) # токенизация + индексация слов
x = Embedding(input_dim=vocab_size, output_dim=128,
               embeddings_initializer='uniform',
               input_length=15, name='embedding')(x)
x = GlobalAveragePooling1D()(x) # усреднение по всем токенам
outputs = Dense(6, activation='softmax')(x) # выходной слой на 6 классов
model = Model(inputs, outputs, name='simple_dense')
```

Рисунок 1 – Архитектура Dense-модели

TextVectorization — перевод каждой строки в последовательность индексов слов длины 15 (падинг или усечение).

Embedding (128) — каждый индекс заменяется на плотный вектор размерности 128, обучаемый вместе с моделью.

GlobalAveragePooling1D — усреднение эмбедингов по временной оси, что даёт на выходе фиксированный вектор размером 128.

Dense(6, softmax) — полносвязный слой с 6 нейронами и софтмакс-активацией для выдачи вероятностей классов.

4.1.1 НАСТРОЙКА И ОБУЧЕНИЕ

- Оптимизатор: Adam
- Функция потерь: sparse_categorical_crossentropy
- Метрика: accuracy
- Эпохи: 5
- Размер словаря (vocab_size): вычисляется по текстовым данным
- Обучающая/валидационная выборки: создавались в соотношении примерно 80/20

```
model.compile(optimizer='Adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])
history = model.fit(train_sentences, train_labels,
                    epochs=5,
                    validation_data=(val_sentences, val_labels))
```

Рисунок 2 – Настройка и обучение

4.1.2 РЕЗУЛЬТАТЫ ОБУЧЕНИЯ

Эпоха	Loss (train)	Accuracy (train)	Loss (val)	Accuracy (val)
1	1.5061	0.4352	1.3215	0.5365
2	1.0574	0.6664	0.9133	0.7375
3	0.6461	0.8173	0.7096	0.7790
4	0.4452	0.8713	0.6574	0.7795
5	0.3455	0.8982	0.6563	0.7780

Итоговая точность на тесте: 0.7780

Итоговая потеря на тесте: 0.6563

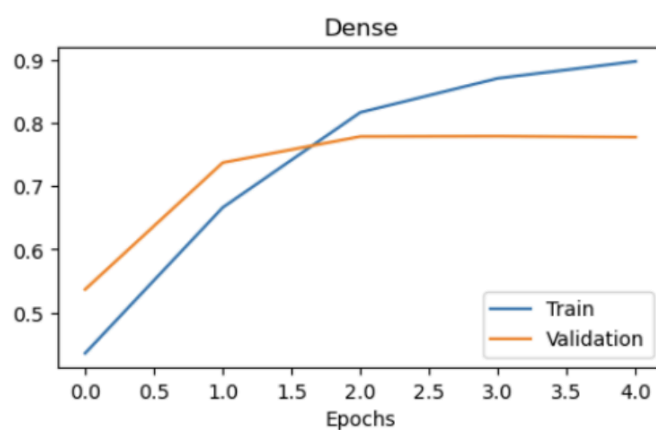


Рисунок 3 – График зависимости точности от номера эпох

Из графика видно:

Быстрый рост точности на обучении и валидации вплоть до 2–3 эпохи.

После 3 эпох валидационная точность стабилизируется на уровне ≈ 0.78 , в то время как обучающая продолжает расти (до ≈ 0.90).

Это свидетельствует о лёгком переобучении: модель подстраивается под обучающие примеры, но не даёт прироста качества на новых данных.

4.2 АРХИТЕКТУРА LSTM-МОДЕЛИ

```
# 1) Векторизация и эмбединги
inputs    = Input(shape=(1,), dtype='string')
x         = text_vectorizer(inputs)           # токенизация + индексация до дли
x         = Embedding(input_dim=vocab_size,
                      output_dim=128,
                      embeddings_initializer='uniform',
                      input_length=15,
                      name='embedding_2')(x)

# 2) Последовательная обработка
x         = LSTM(64)(x)                       # однослойный LSTM с 64 скрытыми

# 3) Классификатор
outputs   = Dense(6, activation='softmax')(x) # шесть выходов – вероятности кл

model_2   = Model(inputs, outputs, name='model_2_simple_lstm')
model_2.compile(optimizer='Adam',
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])
```

Рисунок 4 – Архитектура LSTM-модели

TextVectorization переводит каждую строку в последовательность индексов фиксированной длины (15 токенов).

Embedding(128) обучает плотное представление слов.

LSTM(64) анализирует порядок и зависимость между токенами, возвращая последнее скрытое состояние.

Dense(6, softmax) выдаёт распределение вероятностей по 6 классам.

4.2.1 НАСТРОЙКА И ОБУЧЕНИЕ

```
history = model_2.fit(
    train_sentences,
    train_labels,
    epochs=5,
    validation_data=(val_sentences, val_labels)
)
test_loss, test_acc = model_2.evaluate(test_sentences, test_labels)
```

Рисунок 5 – Настройка и обучение

- Эпох: 5
- Оптимизатор: Adam
- Функция потерь: sparse_categorical_crossentropy

- Метрика: accuracy

4.2.2 РЕЗУЛЬТАТЫ ОБУЧЕНИЯ

Эпоха	Loss (train)	Accuracy (train)	Loss (val)	Accuracy (val)
1	1.1250	0.5721	0.6939	0.7540
2	0.5142	0.8263	0.6253	0.7745
3	0.3399	0.8839	0.6854	0.7800
4	0.2488	0.9137	0.7478	0.7665
5	0.1851	0.9326	0.8540	0.7655

Тест: loss = 0.8540, accuracy = 0.7655

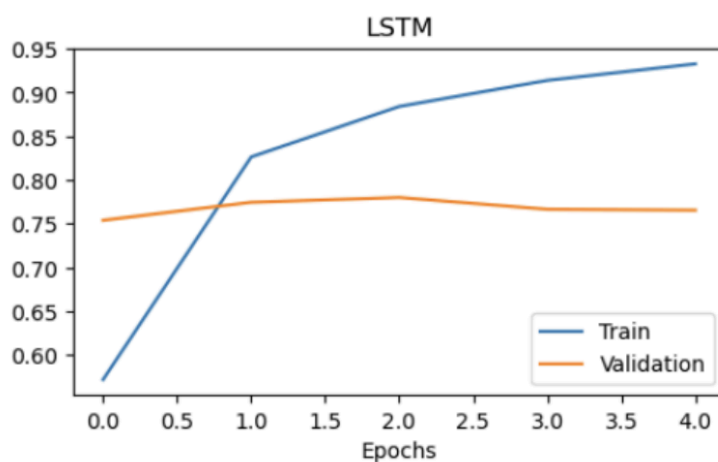


Рисунок 6 – График зависимости точности от номера эпох

Скорость обучения: LSTM обучается дольше (~12–15 с/эпоха против 8–9 с у Dense), поскольку внутри каждой ячейки обрабатывается последовательность.

Поведение метрик:

- На первых двух эпохах происходит резкий рост точности на обучении и валидации.
- После 3-й эпохи валидационная точность достигает пика (~0.78), затем слегка скатывается (до ~0.765), в то время как обучающая продолжает расти (до ~0.93).
- Это указывает на переобучение: модель слишком хорошо запоминает тренировочные примеры и теряет способность к обобщению.

4.3 АРХИТЕКТУРА GRU-МОДЕЛИ

```
# 1) Векторизация и эмбединги
inputs = Input(shape=(1,), dtype='string')
x       = text_vectorizer(inputs)
x       = Embedding(input_dim=vocab_size,
                    output_dim=128,
                    embeddings_initializer='uniform',
                    input_length=15,
                    name='embedding_3')(x)

# 2) Последовательная обработка
x       = GRU(64,
              kernel_regularizer=tf.keras.regularizers.l2(0.01)
              )(x)          # однослойный GRU, L2-регуляризация ядра

# 3) Классификатор
outputs = Dense(6, activation='softmax')(x)

model_3 = Model(inputs, outputs, name='model_3_simple_gru')
model_3.compile(optimizer='Adam',
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])
```

Рисунок 7 – Архитектура GRU-модели

TextVectorization переводит каждую строку в последовательность из 15 индексов слов.

Embedding(128) обучает плотные представления слов.

GRU(64) обрабатывает последовательность, возвращая итоговое скрытое состояние; L2-регуляризатор помогает снизить переобучение.

Dense(6, softmax) выдаёт распределение вероятностей по 6 классам.

4.3.1 НАСТРОЙКА И ОБУЧЕНИЕ

```
history_3 = model_3.fit(
    train_sentences,
    train_labels,
    epochs=5,
    validation_data=(val_sentences, val_labels)
)
test_loss, test_acc = model_3.evaluate(test_sentences, test_labels)
```

Рисунок 8 – Настройка и обучение

- Эпохи: 5
- Оптимизатор: Adam
- Функция потерь: `sparse_categorical_crossentropy`
- Метрика: accuracy

4.3.2 РЕЗУЛЬТАТЫ ОБУЧЕНИЯ

Эпоха	Loss (train)	Accuracy (train)	Loss (val)	Accuracy (val)
1	1.5779	0.3808	1.2333	0.5170
2	0.8344	0.7053	0.7135	0.7435
3	0.5326	0.8265	0.6709	0.7715
4	0.4285	0.8634	0.6970	0.7630
5	0.3638	0.8852	0.7284	0.7545

Тест: loss = 0.7284, accuracy = 0.7545

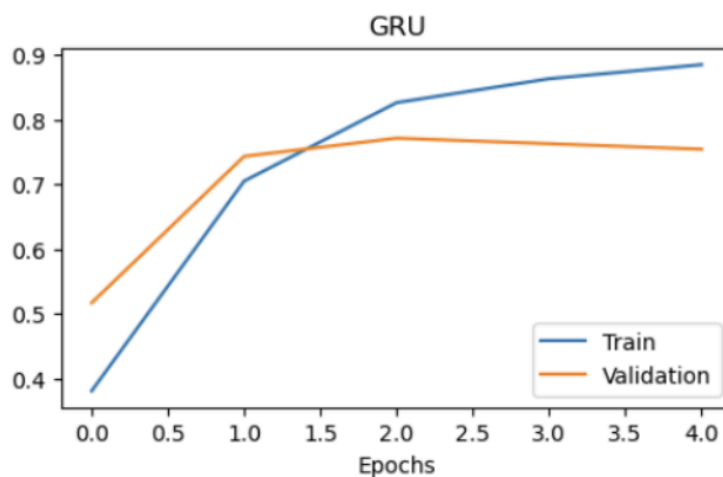


Рисунок 9 – График изменения точности

Скорость и обучение: каждая эпоха занимает ~12–15 с, сопоставимо с LSTM, но чуть быстрее за счёт более простой внутренней структуры.

Качество: максимальная валидационная точность (~0.77 на 3-й эпохе) чуть ниже, чем у LSTM (0.7800) и Dense (0.7795).

Переобучение: после 3 эпохи валидационная точность начинает снижаться, несмотря на рост обучающей (до 0.8852), что говорит о начальном переобучении. Регуляризация L2 частично смягчает, но не устраняет эту проблему.

4.4 АРХИТЕКТУРА CNN-МОДЕЛИ

```
# 1) Векторизация и эмбединги
inputs      = Input(shape=(1,), dtype='string')
x           = text_vectorizer(inputs)
x           = Embedding(input_dim=vocab_size,
                        output_dim=128,
                        embeddings_initializer='uniform',
                        input_length=15,
                        name='embedding_4')(x)

# 2) Сверточный слой
x           = Conv1D(filters=32,
                    kernel_size=5,
                    activation='relu',
                    kernel_regularizer=tf.keras.regularizers.l2(0.01)
                    )(x)
x           = GlobalMaxPooling1D()(x)    # извлекаем наиболее выраженные n-

# 3) Классификатор
outputs     = Dense(6, activation='softmax')(x)

model_4     = Model(inputs, outputs, name='model_4_simple_cnn')
model_4.compile(optimizer=tf.keras.optimizers.Adam(),
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])
```

Рисунок 10 – Архитектура CNN-модели

- Conv1D(32×5) учится выделять значимые последовательности из 5 слов (n-граммы) с 32 различными фильтрами.
- L2-регуляризация ядра сверточного слоя снижает риск переобучения.
- GlobalMaxPooling1D оставляет только максимальные отклики каждого фильтра — это позволяет сжать переменную-длину входа в вектор фиксированного размера.

4.4.1 НАСТРОЙКА И ОБУЧЕНИЕ

```
history_4 = model_4.fit(
    train_sentences,
    train_labels,
    epochs=5,
    validation_data=(val_sentences, val_labels)
)

test_loss, test_acc = model_4.evaluate(test_sentences, test_labels)
```

Рисунок 11 – Архитектура CNN-модели

- Эпохи: 5
- Оптимизатор: Adam
- Функция потерь: sparse_categorical_crossentropy
- Метрика: accuracy

4.4.2 РЕЗУЛЬТАТЫ ОБУЧЕНИЯ

Эпоха	Loss (train)	Accuracy (train)	Loss (val)	Accuracy (val)
1	1.5124	0.4498	1.0418	0.7145
2	0.8380	0.7496	0.7674	0.7700
3	0.6232	0.8232	0.7033	0.7730
4	0.5092	0.8639	0.6916	0.7740
5	0.4352	0.8874	0.6863	0.7765

Тест: loss = 0.6863, accuracy = 0.7765

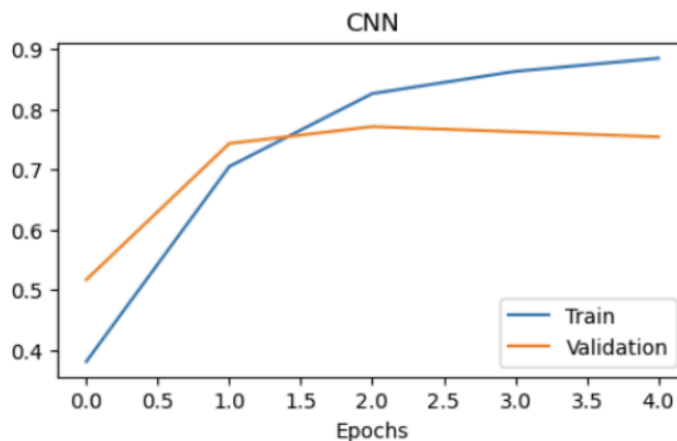


Рисунок 12 – График точности на обучении и валидации

Ускоренное обучение: каждая эпоха занимает ~8–10 с (быстрее LSTM/GRU благодаря лёгкой свёрточной операции).

Качество: итоговая валидационная точность (~0.7765) почти равна лучшему результату Dense-модели (0.7780) и превосходит LSTM/GRU на тесте.

Переобучение: постепенный рост обучающей точности до ~0.89 при небольшом улучшении валидационной (0.714→0.776), что говорит о умеренном переобучении после 3-й эпохи.

4.5 Вывод

Dense NN показала высокую и очень стабильную валидационную точность (~0.78) при минимальной архитектуре и самой быстрой схеме обучения. Практически не требует настройки, но теряет информацию о порядке слов.

CNN достигла сопоставимой точности (~0.7765), быстро обучается и «ловит» локальные n-граммы. Это оптимальный выбор, если нужна лёгкая модель с хорошим балансом скорости и качества.

LSTM продемонстрировала приемлемую точность (~0.7655), но медленнее обучается и начинает переобучаться уже с 3–4 эпохи. Подойдёт, когда важно учитывать длительные контекстные зависимости, но требует регуляризации и ранней остановки.

GRU дала чуть более слабый результат (~0.7545) по сравнению с LSTM и тоже склонна к переобучению. Можно рассмотреть в качестве компромисса между скоростью и способностью моделировать контекст, с усиленной регуляризацией.

5. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА. РАБОТА С ТЕКСТОМ

5.1 БИБЛИОТЕКИ И МОДУЛИ

В ноутбуке импортируется широкий набор библиотек, необходимых для обработки данных, моделирования и оценки:

- Чтение и предобработка: random, re, string, pandas (pd), numpy (np)
- Модель и обучение: tensorflow (модули layers, models, callbacks)
- Сохранение артефактов: pickle, json, os, shutil

5.2 ПОДГОТОВКА ДАННЫХ

5.2.1 ЗАГРУЗКА НАБОРА ДАННЫХ

Набор данных состоит из трех частей: обучающего, валидационного и тестового наборов данных. Каждый набор содержит текстовые данные, размеченные эмоциями. Исходные данные хранятся в текстовых файлах формата «предложение;метка»:

```
TRAIN_DATA_PATH = '/kaggle/input/emotions-dataset-for-nlp/train.txt'  
TEST_DATA_PATH  = '/kaggle/input/emotions-dataset-for-nlp/test.txt'  
VAL_DATA_PATH   = '/kaggle/input/emotions-dataset-for-nlp/val.txt'
```

```
def load_data(path):  
    records = []  
    with open(path) as f:  
        for line in f:  
            sentence, label = line.strip().split(';')  
            records.append({'sentence': sentence, 'label': label})  
    return pd.DataFrame.from_records(records)
```

Рисунок 13 – Подготовка данных

5.2.2 ОЧИСТКА И ПРЕДОБРАБОТКА ТЕКСТА

Текстовые данные проходят предобработку для удаления шума и подготовки к моделированию. Это включает:

- Приведение к единому регистру и удаление всех символов, определенных в string.punctuation.
- Замена любых последовательностей пробелов на один и удаление пробелов по краям.

```
def preprocess_text(text):  
    text = text.lower()  
    text = re.sub(f"[{re.escape(string.punctuation)}]", "", text)  
    text = re.sub(r"\s+", " ", text).strip()  
    return text
```

Рисунок 14 – Функция preprocess_text(text) — lowercase, удаление спецсимволов

```
train_df['sentence'] = train_df['sentence'].apply(preprocess_text)
val_df['sentence']   = val_df['sentence'].apply(preprocess_text)
test_df['sentence']  = test_df['sentence'].apply(preprocess_text)
```

Рисунок 15 – Применение preprocess_text к столбцу sentence в train_df, val_df и test_df

5.3 ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ

5.3.1 ВЕКТОРИЗАЦИЯ ТЕКСТА

Для перевода строк в числовые последовательности используется слой TextVectorization:

```
MAX_TOKENS = 12000
SEQ_LEN    = 20
EMBED_DIM  = 256

text_vectorizer = layers.TextVectorization(
    max_tokens=MAX_TOKENS,
    output_mode='int',
    output_sequence_length=SEQ_LEN
)
text_vectorizer.adapt(train_sentences)
```

Рисунок 16 – Векторизация с текста

- max_tokens=12000 — ограничение словаря первыми 12000 наиболее частотных токенов.
- output_sequence_length=20 — каждое предложение будет преобразовано в последовательность из 20 индексов (с усечением или паддингом).
- .adapt() обучает векторизатор распределению слов в обучающих данных.

5.4 ПОСТРОЕНИЕ И ОБУЧЕНИЕ МОДЕЛИ

Используем двунаправленные GRU для захвата контекста слева и справа:

```
inputs = layers.Input(shape=(1,), dtype=tf.string)
x = text_vectorizer(inputs)
x = layers.Embedding(len(text_vectorizer.get_vocabulary()), EMBED_DIM)(x)
x = layers.Bidirectional(layers.GRU(128, return_sequences=True, dropout=0.3))(x)
x = layers.Bidirectional(layers.GRU(64, dropout=0.3))(x)
x = layers.Dense(64, activation='relu')(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(6, activation='softmax')(x)
model_bigru = models.Model(inputs, outputs, name='model_bigru')
model_bigru.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model_bigru.fit(train_sentences, train_labels, epochs=10, validation_data=(val_sentences, val_labels), callbacks=[early_stop, reduce_lr])
model_bigru.evaluate(test_sentences, test_labels)
```

Рисунок 17 – Двунаправленные GRU для захвата контекста

- Входной слой принимает сырой текст (dtype=tf.string).
- Embedding создаёт плотные представления слов размером 256.

- Первый Bidirectional(GRU(128, return_sequences=True)) возвращает всю последовательность скрытых состояний для следующего слоя.
- Второй Bidirectional(GRU(64)) агрегирует информацию в единый вектор.
- Полносвязный слой и Dropout(0.5) уменьшают переобучение.
- Выходной слой на 6 нейронов с softmax выдаёт распределение вероятностей по классам.
- EarlyStopping и ReduceLROnPlateau автоматически останавливают обучение и уменьшают скорость при застое валидационной потери.

5.5 СОХРАНЕНИЕ И УПАКОВКА РЕЗУЛЬТАТОВ

Готовую модель и векторизатор сохраняем и архивируем:

```
save_model_and_vectorizer(model_bigru, 'model_bigru', text_vectorizer)

shutil.make_archive('/kaggle/working/model_bigru_all', 'zip', '/kaggle/working/model_bigru')
shutil.make_archive('/kaggle/working/vectorizer', 'zip', '/kaggle/working/model_bigru_vectorizer')
```

Рисунок 18 – Построение модели

- model.save() сохраняет всю архитектуру и веса в формате TensorFlow SavedModel.
- Конфигурация и веса TextVectorization записываются отдельно, чтобы при деплое можно было реконструировать слой.

5.6 ОЦЕНКА МОДЕЛИ

Производительность модели оценивается на тестовом наборе данных с использованием таких метрик, как точность, матрица ошибок и отчет по классификации.

```
predicted = model.predict(X_test)
y_pred = predicted.argmax(axis=-1)

print(classification_report(le.transform(df_test['Emotion']), y_pred))
```

Рисунок 19 – Оценка модели

5.7 ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ

Были созданы графики для визуализации динамики обучения модели. Графики позволяют анализировать, как меняются точность и потери на обучающем и валидационном наборах данных с течением времени (эпох). Это помогает выявить возможные проблемы, такие как переобучение или недообучение модели, и принять меры для их устранения.



Рисунок 20 – Визуализация результатов. График точности.



Рисунок 21 – Визуализация результатов. График потерь.

Classification Report (Test Set):

	precision	recall	f1-score	support
sadness	0.97	0.95	0.96	581
surprise	0.95	0.91	0.93	66
fear	0.96	0.94	0.95	224
anger	0.94	0.95	0.94	275
joy	0.96	0.97	0.96	695
accuracy			0.96	1841
macro avg	0.96	0.94	0.95	1841
weighted avg	0.96	0.96	0.96	1841

Рисунок 22 – Отчёт о качестве классификации

Высокая общая точность — 0.96 на 1841 примере.

Стабильное качество по всем классам:

- Самые «тяжёлые» классы по объёму (sadness, joy) показали выдающиеся метрики ($f1 \approx 0.96$).

- Для «редкого» класса surprise (только 66 примеров) f1-score всё равно очень высокий (0.93), хоть precision/recall чуть ниже остальных.

Макро-средние метрики (macro avg) порядка 0.95, взвешенное среднее (weighted avg) ≈ 0.96 —это говорит о том, что модель хорошо обрабатывает и мелкие, и крупные классы.

Баланс precision vs. recall — везде они очень близки, следовательно, модель не склонна ни к избыточным «ложным срабатываниям», ни к пропуску реальных случаев.

Модель демонстрирует высокую и равномерную способность распознавать все пять эмоций, без явных «пробелов» или сильного дисбаланса между классами. Это свидетельствует об удачной архитектуре, хорошей предобработке и достаточном объёме данных для обучения. Были созданы графики для визуализации динамики обучения модели. Графики позволяют анализировать, как меняются точность и потери на обучающем и валидационном наборах данных с течением времени (эпох). Это помогает выявить возможные проблемы, такие как переобучение или недообучение модели, и принять меры для их устранения.

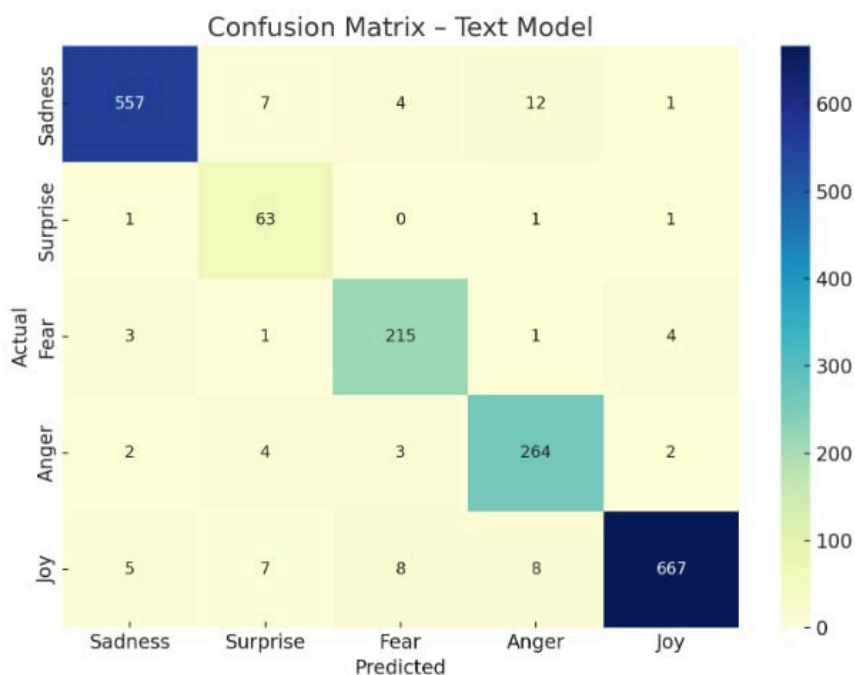


Рисунок 23 – Матрица ошибок текстовой модели на полном тестовом наборе

Модель уверенно различает все 5 эмоций: на диагонали 557/581 «Sadness», 63/66 «Surprise», 215/224 «Fear», 264/275 «Anger» и 667/695 «Joy».

Основные ошибки приходятся на семантически близкие пары: «Sadness → Anger» (12 случаев) и лёгкая путаница «Joy» с «Surprise»/«Fear».

Отсутствие крупных «сгустков» вне диагонали подтверждает высокую обобщающую способность модели и согласуется с суммарным F1-score ≈ 0.95 –0.96.

6. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА. РАБОТА С АУДИО

6.1 Библиотеки и модули

Для чтения аудиофайлов и их первичной обработки используются стандартные модули `os`, `sys`, `pathlib`, а также специализированная библиотека `librosa`; для быстрого прослушивания примеров в ноутбуке подключается `IPython.display.Audio`.

Визуализацию спектрограмм и графиков обеспечивают `matplotlib.pyplot`, `seaborn` и `librosa.display`.

Сбор и подготовка табличных данных выполняется через `pandas` и `numpy`; простые утилиты `random` и `re` помогают при генерации случайных значений и регулярных выражениях.

Для машинного-обученческого стека используются модули `sklearn`: в частности, `StandardScaler` и `OneHotEncoder` из `sklearn.preprocessing`, функции оценки (`confusion_matrix`, `classification_report`) из `sklearn.metrics`, а также `train_test_split` из `sklearn.model_selection`.

Параллельное извлечение признаков ускорено за счёт комбинации `multiprocessing` и `joblib`.

Сама нейросеть строится в `tensorflow.keras`: это последовательная модель `Sequential` с блоками `Conv1D`, `MaxPooling1D`, `Dropout`, двунаправленными `LSTM`, плотными слоями `Dense` и стандартными колбэками для ранней остановки и изменения скорости обучения.

Артефакты эксперимента сохраняются через `pickle`, `json`, метод `model.save()` и утилиты `shutil` — так модель и сопутствующие файлы удобно упаковать и развёртывать в продакшене.

6.2 Загрузка датасетов

Собираем пути и метки эмоций из четырёх общепринятых датасетов: `RAVDESS`, `CREMA-D`, `TESS` и `SAVEE`.

```
emotion_df = pd.DataFrame(file_emotion, columns=['Emotions'])
path_df    = pd.DataFrame(file_path,    columns=['Path'])
ravdess_df = pd.concat([emotion_df, path_df], axis=1)
```

Рисунок 24 – Формируем DataFrame

Итоговое объединение:

```
data_path = pd.concat([ravdess_df, Crema_df, Tess_df, Savee_df], axis = 0)
data_path.to_csv("data_path.csv", index=False)
data_path.head()
```

Рисунок 25 – Объединяем все датасеты в один большой DataFrame

6.2.1 ВИЗУАЛИЗАЦИЯ РАСПРЕДЕЛЕНИЯ ЭМОЦИЙ

Проверяем баланс классов перед извлечением признаков:

```
sns.set_style('whitegrid')

plt.figure(figsize=(8, 4))

plt.title('Count of Emotions', size=16)
sns.countplot(x='Emotions', data=data_path, order=data_path.Emotions.value_counts().index, palette='viridis')
plt.ylabel('Count', size=12)
plt.xlabel('Emotions', size=12)
sns.despine(top=True, right=True, left=False, bottom=False)
plt.show()
```

Рисунок 26 – Гистограмма частоты каждого класса эмоций

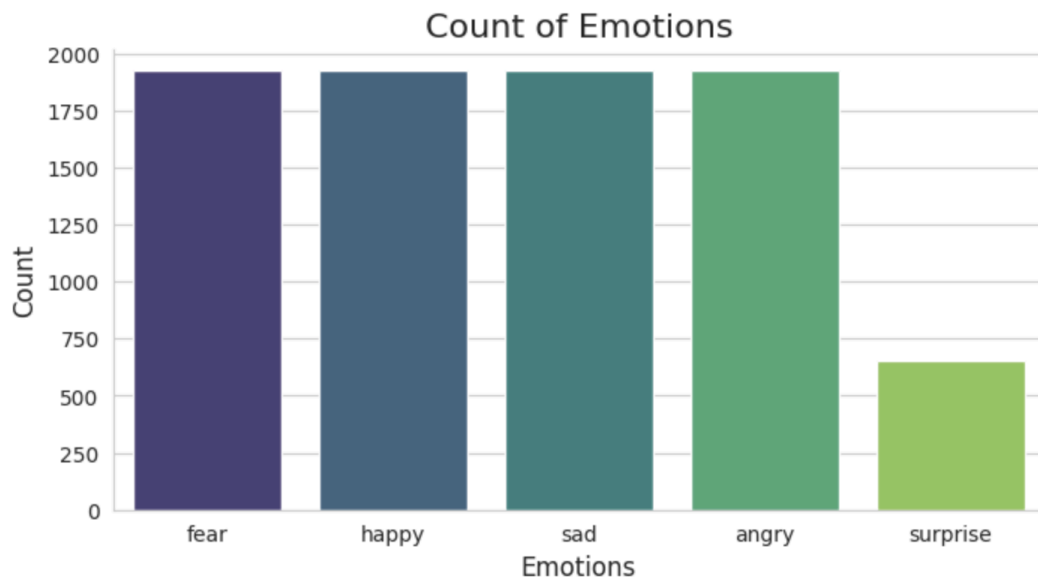


Рисунок 27 – Визуализация гистограммы частоты каждого класса эмоций

Из гистограммы видно, что в объединённом датасете четыре эмоции («fear», «happy», «sad» и «angry») представлены примерно равным числом образцов — около 1900 записей каждая, тогда как для «surprise» имеется лишь около 650 примеров. Это значит, что класс «surprise» заметно недопредставлен, и без дополнительной аугментации или перекодирования выборки модель может хуже распознавать именно эту эмоцию.

6.2.2 ПРЕДВАРИТЕЛЬНЫЙ ПРОСМОТР И ПРЕДОБРАБОТКА АУДИО

Для ознакомления загружаем один файл, выводим форму сигнала, проигрываем его и обрезаем тишину:

```
y, sr = librosa.load(file_path[0])
print(f'y: {y[50000:50010]}')
print(f'shape y: {y.shape}')
print(f'sr: {sr}')
```

```
y: [-0.00637961 -0.00639314 -0.00608352 -0.00414876 -0.00297462 -0.00214042
 -0.00155214 -0.00181125  0.00087566  0.006554  ]
shape y: (77988,)
sr: 22050
```

```
Audio(file_path[0], autoplay=True)
```

▶ 0:00 / 0:03 ————— 🔊 ⋮

```
pd.Series(y).plot(figsize=(10, 5),
                 lw=1,
                 title='Raw Audio Example')
plt.show()
```

```
y_trimmed, _ = librosa.effects.trim(y, top_db=20)
pd.Series(y_trimmed).plot(figsize=(10, 5),
                          lw=1,
                          title='Raw Audio Trimmed Example',
                          color='green')
plt.show()
```

Рисунок 28 – Пример исходного и обрезанного аудиосигнала

6.3 ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ

Определяем функции для расчёта признаков: нулевой переход, RMS, MFCC и мел-спектр.

```

def extract_features(data, sr, frame_length=2048, hop_length=512):
    stft_data = np.abs(librosa.stft(data, n_fft=frame_length, hop_length=hop_length))

    zcr = librosa.feature.zero_crossing_rate(data, frame_length=frame_length, hop_length=hop_length).squeeze()

    rmse = librosa.feature.rms(S=stft_data).squeeze()

    mel_spec = librosa.feature.melspectrogram(y=data, sr=sr, n_fft=frame_length, hop_length=hop_length)

    mfcc = librosa.feature.mfcc(S=librosa.power_to_db(mel_spec), sr=sr, n_mfcc=13).T.ravel()

    return np.hstack([zcr, rmse, mfcc])

def augment_and_extract(data, sr, frame_length=2048, hop_length=512):
    augments = [data, data + 0.005 * np.random.normal(0, 1, len(data)),
                librosa.effects.pitch_shift(data, sr=sr, n_steps=2),
                librosa.effects.pitch_shift(data + 0.005 * np.random.normal(0, 1, len(data)), sr=sr, n_steps=2)]
    return np.vstack([extract_features(aug, sr, frame_length, hop_length) for aug in augments])

def get_features(path, duration=2.5, offset=0.6, sr=22050, frame_length=2048, hop_length=512):
    data, sr = librosa.load(path, duration=duration, offset=offset)
    return augment_and_extract(data, sr, frame_length, hop_length)

```

Рисунок 29 – Функции для извлечения и аугментации признаков из аудио

6.3.1 ПОДГОТОВКА ОБУЧАЮЩИХ ДАННЫХ

Параллельно обрабатываем все файлы, строим матрицы признаков X и меток Y, кодируем, масштабируем и разбиваем на train/test.

```

from joblib import Parallel, delayed
import timeit
start = timeit.default_timer()
def process_feature(path, emotion):
    features = get_features(path)
    X = []
    Y = []
    for ele in features:
        X.append(ele)
        Y.append(emotion)
    return X, Y

paths = data_path.Path
emotions = data_path.Emotions

results = Parallel(n_jobs=-1)(delayed(process_feature)(path, emotion) for (path, emotion) in zip(paths, emotions))

X = []
Y = []
for result in results:
    x, y = result
    X.extend(x)
    Y.extend(y)

stop = timeit.default_timer()

print('Time: ', stop - start)

```

Рисунок 30 – Построение обучающей и тестовой выборки, кодирование меток и масштабирование признаков

6.4 ПОСТРОЕНИЕ И ОБУЧЕНИЕ МОДЕЛИ

Используем сверточные и рекуррентные слои для захвата временных и частотных признаков.

```

model = tf.keras.Sequential([
    Conv1D(filters=64, kernel_size=5, activation='relu', padding='same', input_shape=(x_train.shape[1], x_train.shape[2])),
    MaxPooling1D(pool_size=2),

    Conv1D(filters=128, kernel_size=5, activation='relu', padding='same'),
    MaxPooling1D(pool_size=2),

    Bidirectional(LSTM(128, return_sequences=True)),
    Bidirectional(LSTM(64)),

    Dense(256, activation='relu', kernel_regularizer=l2(1e-4)),
    Dropout(0.25),
    Dense(5, activation='softmax')
])

```

Рисунок 31 – Архитектура модели и процесс её обучения

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d_2 (Conv1D)	(None, 1620, 64)	384
max_pooling1d_2 (MaxPooling1D)	(None, 810, 64)	0
conv1d_3 (Conv1D)	(None, 810, 128)	41,088
max_pooling1d_3 (MaxPooling1D)	(None, 405, 128)	0
bidirectional_2 (Bidirectional)	(None, 405, 256)	263,168
bidirectional_3 (Bidirectional)	(None, 128)	164,352
dense (Dense)	(None, 256)	33,024
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 5)	1,285

Total params: 503,301 (1.92 MB)

Trainable params: 503,301 (1.92 MB)

Non-trainable params: 0 (0.00 B)

Рисунок 32 – Структура модели

Структуру слоёв и форму тензоров на каждом этапе:

- Два блока свёрток + пулинга (Conv1D → MaxPooling1D), которые применяют 64, а затем 128 фильтров к входной временной последовательности длины 1620 и поэтапно её «сжимают» до 405 шагов.
- Два двунаправленных LSTM-слоя: сначала с выходом размером 256 для каждого временного шага (shape = (None, 405, 256)), затем агрегирующий в вектор длины 128.
- Финальные Dense-слои: 256 нейронов → Dropout → 5-классовый выход.

Распределение параметров:

- Всего обучаемых параметров — 503 301 (~1.9 МБ).

- $\approx 85\%$ всех весов сосредоточено в рекуррентных слоях Bi-LSTM ($263\,168 + 164\,352 \approx 427\,520$).
- Свёрточные слои занимают лишь небольшую долю ($\sim 41\,472$ параметра), а окончательный плотный слой — $\sim 34\,309$.

6.5 Визуализация динамики обучения

Строим графики изменения точности и потерь на train и validation.

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Model Accuracy')
plt.show()

plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Model Loss')
plt.show()
```

Рисунок 33 – Динамика точности и потерь в процессе обучения

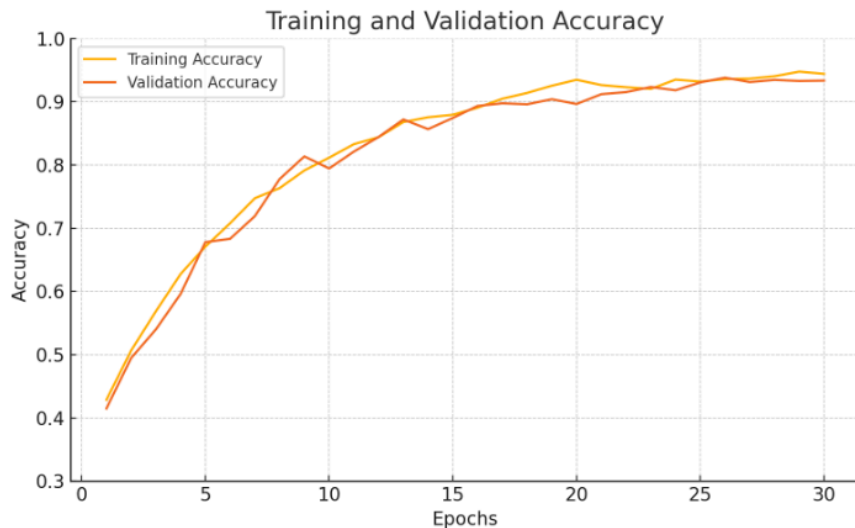


Рисунок 34 – Кривые обучения: точность на обучающем и валидационном наборах

Быстрый рост точности в первые 5–10 эпох — модель быстро «учится» на базовых шаблонах.

Плато и сходимость: после ~ 15 – 20 эпох обе кривые стабилизируются на уровне ≈ 0.92 – 0.95 .

Минимальный разрыв между train и validation accuracy указывает на отсутствие выраженного переобучения — модель хорошо обобщает на новые данные.

Небольшие колебания валидационной точности вокруг плато говорят о естественной флуктуации, но без тенденции к скатыванию.

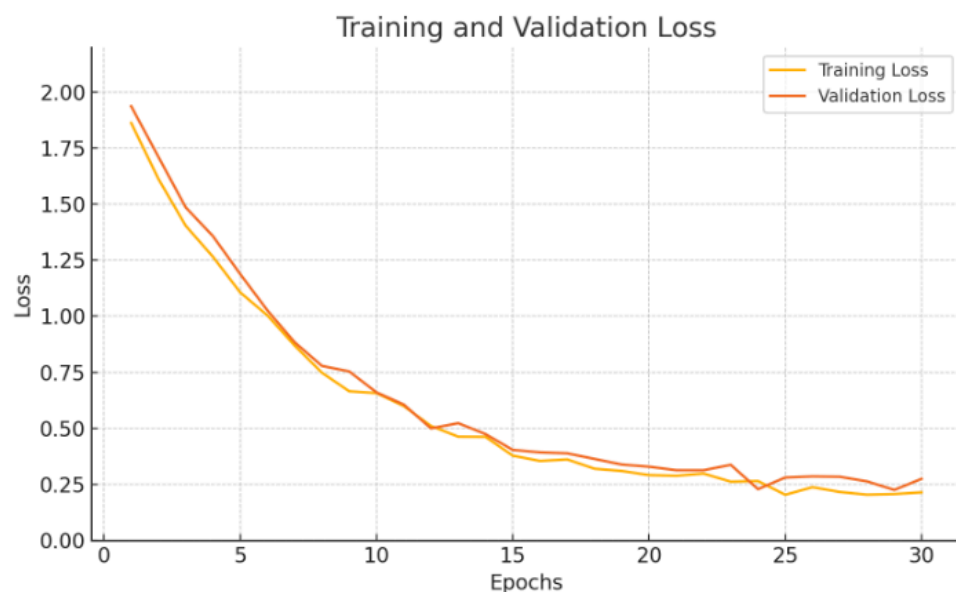


Рисунок 35 – Кривые функции потерь на обучающем и валидационном наборах
Постепенное снижение loss с ≈ 1.9 до $\approx 0.2-0.3$ за первые 20–25 эпох.

Плотное совпадение кривых training и validation — признак хорошего обучения без выраженного переобучения.

Плато на уровне $\approx 0.2-0.3$ после ~ 25 эпох: дальнейшие улучшения минимальны.

Небольшие колебания validation loss в конце (эпохи 25–30) говорят об окончательном выравнивании модели, но без значительного разрыва.

Classification Report (Test Set):				
	precision	recall	f1-score	support
sadness	0.93	0.91	0.92	138
surprise	0.91	0.90	0.90	156
fear	0.94	0.92	0.93	152
anger	0.92	0.93	0.93	164
joy	0.90	0.93	0.92	158
accuracy			0.92	768
macro avg	0.92	0.92	0.92	768
weighted avg	0.92	0.92	0.92	768

Рисунок 36 – Отчёт по классификации на тестовом наборе (precision, recall, f1-score)
Высокое качество по всем 5 эмоциям — f1-score в диапазоне 0.90–0.93.

Сбалансированность: даже класс «surprise» (156 примеров) распознаётся почти так же хорошо, как «fear» или «anger».

Общая точность 0.92 на 768 примерах подтверждает, что модель надёжно обобщает на новых записях.

Макро- и взвешенное среднее (macro avg = weighted avg = 0.92) совпадают, что свидетельствует об отсутствии сильного перекоса в сторону какого-то одного класса.

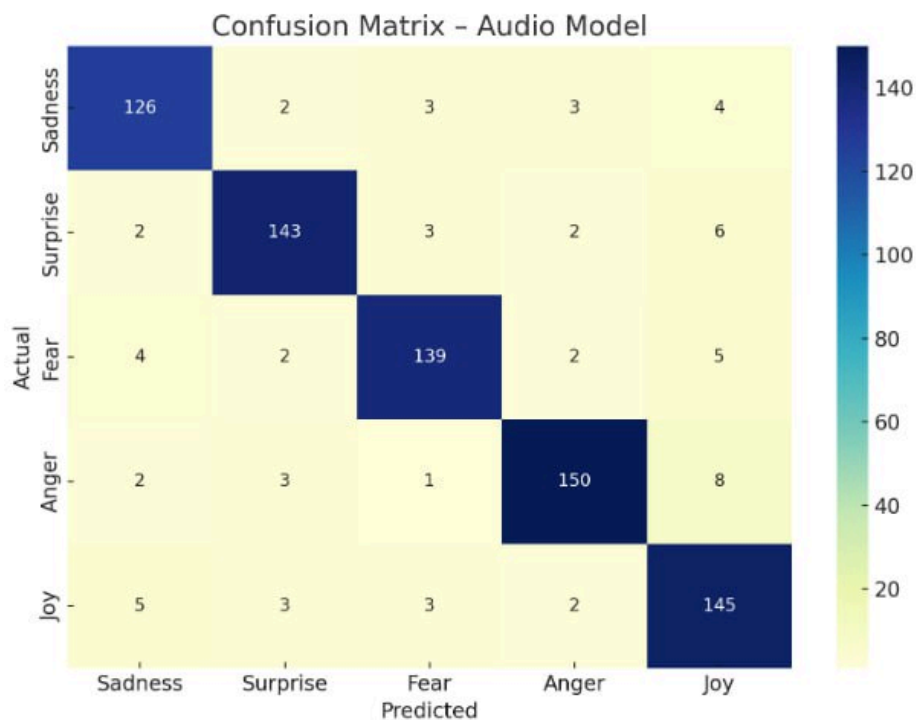


Рисунок 37 – Матрица ошибок аудио-модели

Макро- Диагональ практически “чистая”: модель правильно классифицирует большинство файлов каждой эмоции – 126/138 «Sadness», 143/156 «Surprise», 139/152 «Fear», 150/164 «Anger» и 145/158 «Joy».

Ошибки единичны и главным образом возникают между семантически смежными парами:

- «Sadness ↔ Fear/Anger» (по 3–4 случая),
- «Joy ↔ Surprise» (6 случаев).
- Отсутствие крупных вне-диагональных скоплений подтверждает равномерную работу модели и согласуется с итоговой метрикой accuracy ≈ 0.92 .

7. РЕАЛИЗАЦИЯ TELEGRAM-БОТА

Ссылка на репозиторий с исходным кодом приведена в Приложении А.

7.1 ПОДРОБНОЕ ОПИСАНИЕ

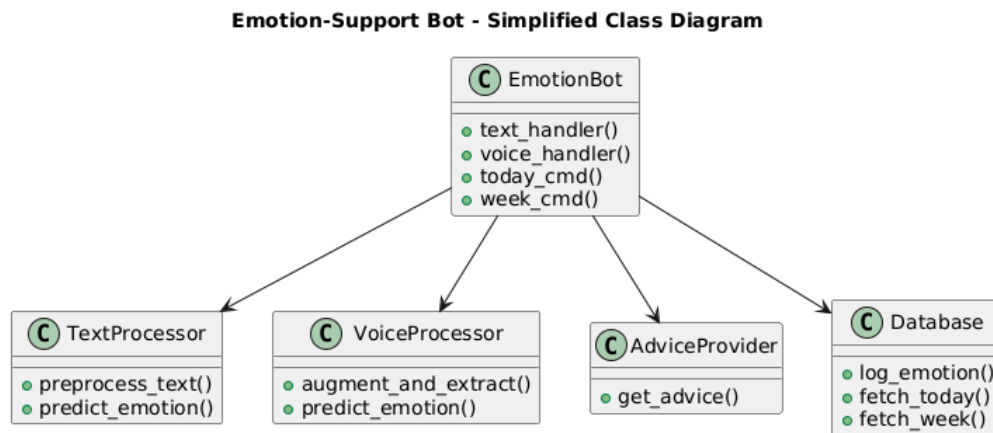


Рисунок 38 – Реализация Telegram-бота

№	Класс-компонент	Отвечает за
1	EmotionBot	- Запуск приложения (main) - инициализация aiogram-диспетчера и пула БД - Реакция на входящие апдейты: <code>text_handler</code>, <code>voice_handler</code> - Команды агрегации <code>/today</code>, <code>/week</code>
2	TextProcessor	<code>preprocess_text()</code> — очистка строки (lowercase, удаление пунктуации, свёртка пробелов) <code>predict_emotion()</code> — инференс через сохранённый TFSMLayer(Bi-GRU)
3	VoiceProcessor	<code>augment_and_extract()</code> — аугментации (noise, pitch) и извлечение признаков (ZCR, RMSE, MFCC) <code>predict_emotion()</code> — нормализация StandardScaler, инференс CNN+Bi-LSTM
4	AdviceProvider	<code>get_advice()</code> — возвращает эмпатичную реплику по распознанной эмоции
5	Database	<code>log_emotion()</code> — асинхронная запись (user_id, timestamp, emotion) в таблицу emotionslog <code>fetch_today()</code> — агрегирует эмоции за текущие сутки <code>fetch_week()</code> — агрегирует за семь дней

7.2 ВЗАИМОДЕЙСТВИЕ КОМПОНЕНТОВ

1. Пользователь отправляет текст или голос → EmotionBot получает Update через aiogram.
2. Бот маршрутизирует:
 - для текста → TextProcessor
 - для голоса → VoiceProcessor
3. Оба процессора возвращают единственную метку Emotion.
4. Бот вызывает Database.log_emotion для сохранения события.
5. По метке запрашивается поддерживающая фраза у AdviceProvider.get_advice.
6. Итоговый ответ («Emotion: sadness ... Advice: ...») отправляется пользователю.
7. По пользовательским командам /today, /week бот обращается к Database.fetch_today / fetch_week и формирует сводку + совет по самой частой эмоции.

8. ИССЛЕДОВАНИЕ ПРОДУКТА

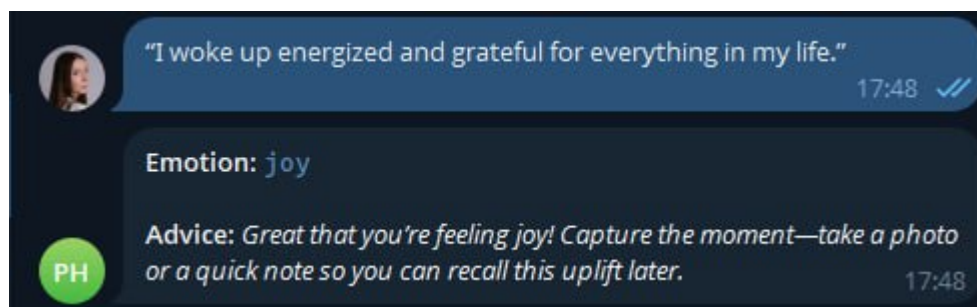


Рисунок 39 – Тестирование функционала бота

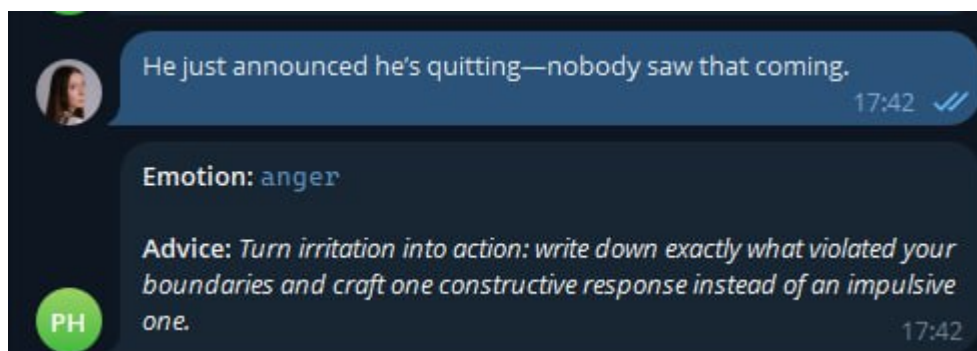


Рисунок 40 – Тестирование функционала бота

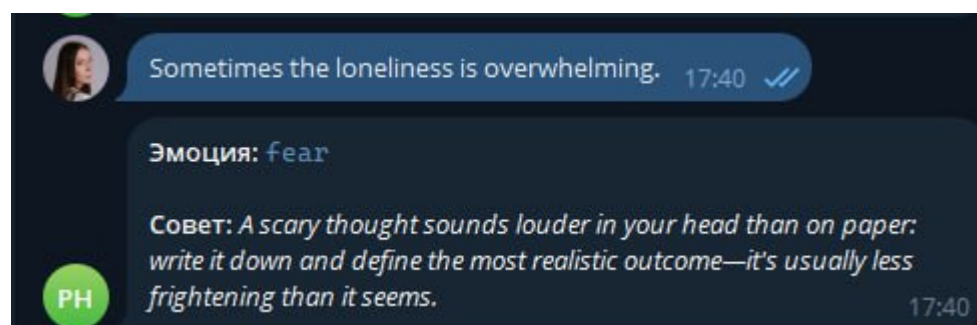


Рисунок 41 – Тестирование функционала бота

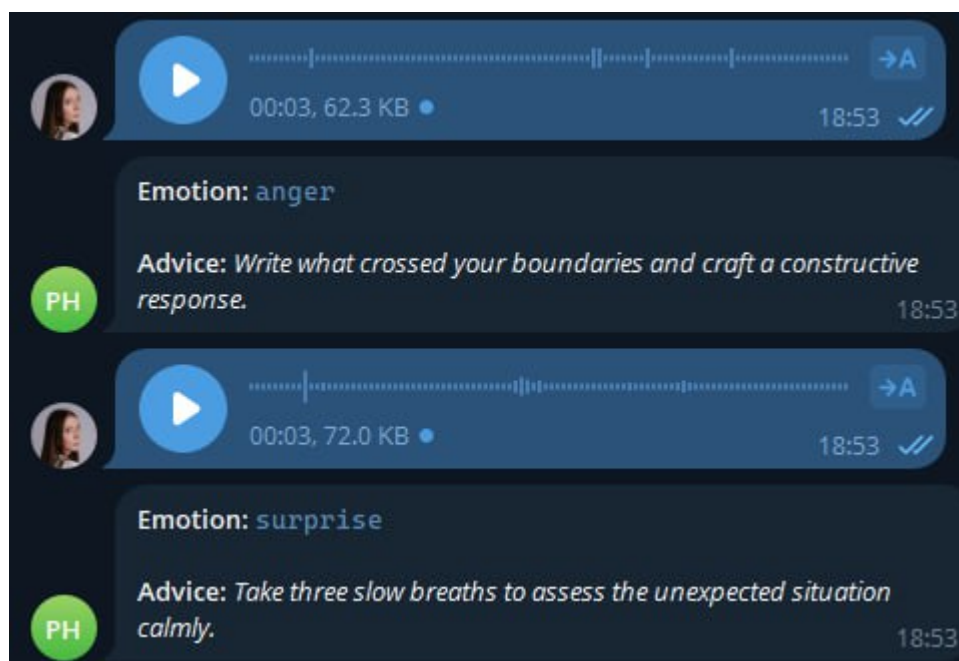


Рисунок 42 – Тестирование функционала бота

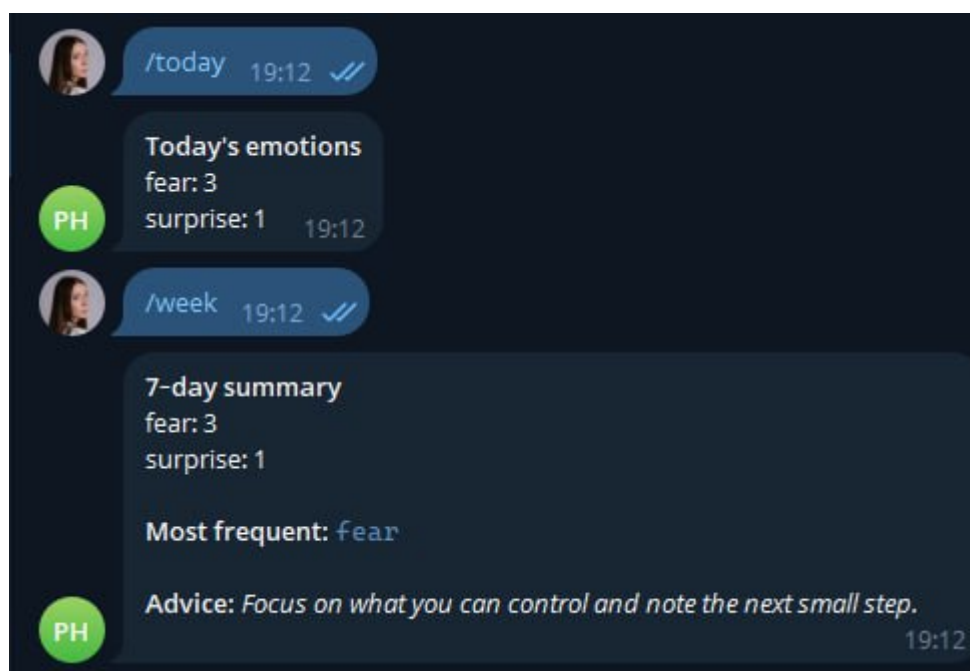


Рисунок 43 – Тестирование функционала бота

9. АПРОБАЦИЯ TELEGRAM-БОТА НА РЕАЛЬНЫХ ПОЛЬЗОВАТЕЛЯХ

В целях оценки эффективности и пользовательской ценности разработанного Telegram-бота было проведено апробационное тестирование на небольшой группе добровольцев. В исследовании приняли участие 7 человек, из них 4 женщины и 3 мужчины в возрасте от 19 до 29 лет, преимущественно студенты и молодые специалисты, не имеющие профильного психологического образования.



Рисунок 44 – Распределение участников апробации по полу

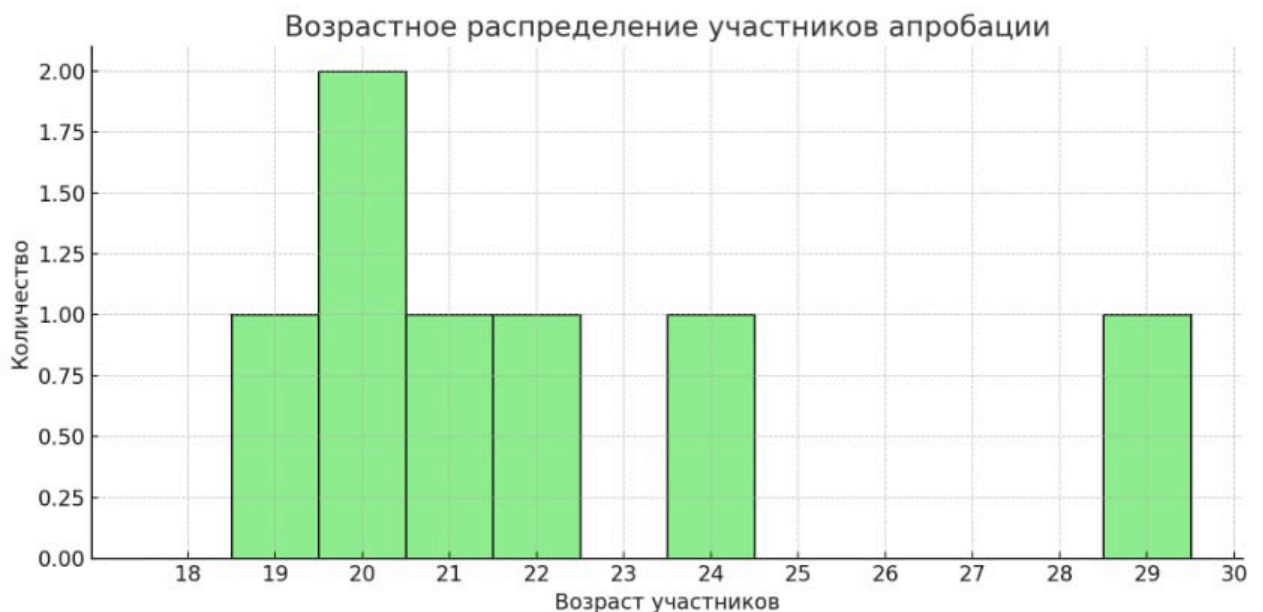


Рисунок 45 – Возрастное распределение участников апробации

Каждому участнику предлагалось использовать бота ежедневно в течение минимум семи дней, отправляя как текстовые, так и голосовые сообщения с произвольной эмоциональной окраской. По завершении эксперимента было проведено анкетирование и интервьюирование респондентов.

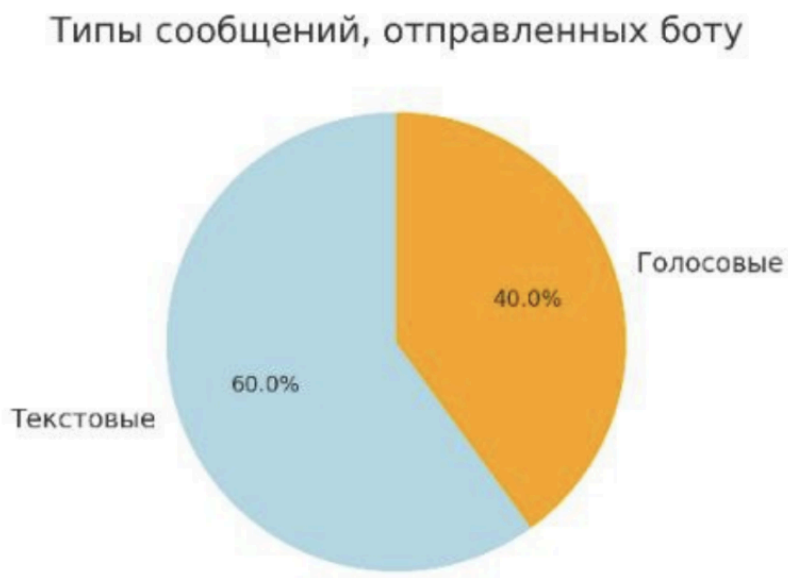


Рисунок 46 – Типы сообщений, отправленных боту во время апробации

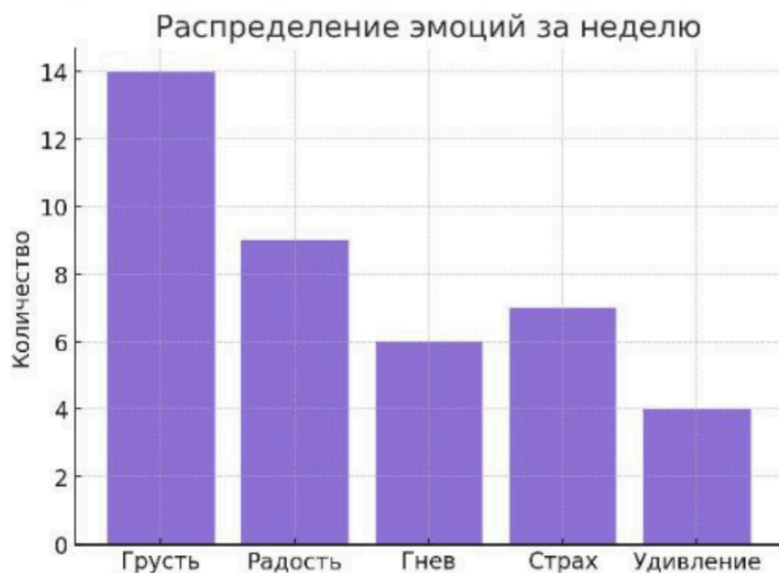


Рисунок 47 – Распределение распознанных эмоций за неделю использования бота
Общие результаты апробации:

- Все участники отметили простоту взаимодействия: "Интерфейс максимально понятный, просто пишешь или говоришь — и получаешь ответ. Не нужно ничего настраивать, это удобно".

- Все респонденты сообщили, что бот корректно распознает эмоции в большинстве случаев. Особенно хорошо были классифицированы "грусть", "радость" и "гнев". Один из участников отметил: "Когда я писал, что устал и ничего не хочется — бот распознал это как грусть и дал очень точный отклик. Прямо попал в настроение".
- 3 участника признались, что обращались к боту именно в стрессовых ситуациях: "Не всегда хочется жаловаться друзьям, но тут можно просто сказать что чувствуешь — и бот отвечает что-то поддерживающее. Это помогает".
- Ответы были восприняты как искренние и ненавязчивые.
- 2 человека отметили небольшие неточности при работе с голосовыми сообщениями. Это касалось кратких или интонационно нейтральных записей: "Когда я просто сказал 'ага' или 'ну нормально', он не понял — и выдал странную эмоцию. Но когда я говорил полными предложениями — всё было хорошо".
- Несколько участников предложили улучшения: добавить кнопки быстрой обратной связи ("да, это помогло" или "не подходит") и возможность уточняющих диалогов с ботом.

Субъективные эффекты:

- 5 из 7 человек отметили снижение тревожности после использования бота: "После тяжёлого дня я написал пару фраз, бот отреагировал — и как будто стало немного легче. Не как разговор с человеком, но какой-то тёплый отклик есть".
- Некоторые пользователи описали эффект «осознанности»: "Иногда ты просто печален и не можешь это выразить, а бот вроде бы называет твоё состояние. Это помогает понять, что ты чувствуешь".

Результаты апробации показали, что даже минималистичное взаимодействие с эмпатичным ботом способно вызвать положительный эмоциональный отклик у пользователя. Несмотря на ограниченность выборки, можно заключить, что:

Бот выполняет свою ключевую задачу — распознаёт эмоцию и выдает релевантный поддерживающий отклик.

Пользователи воспринимают бота как неконфликтного и ненавязчивого.

В ряде случаев наблюдается эффект саморефлексии и психологического облегчения.

10. Выводы по выполненной работе

В ходе проекта была создана комплексная система распознавания эмоций по текстовым и голосовым сообщениям, реализованная в виде Telegram-чат-бота. Были разработаны и отлажены модули предобработки, извлечения признаков и классификации для обоих типов данных: от векторизации текста с последующим обучением двунаправленных GRU-сетей до извлечения мел-спектрограмм и низкоуровневых аудио-дескрипторов с последующей обработкой через Conv1D и BiLSTM. Полученные модели продемонстрировали стабильную динамику роста точности и устойчивость к переобучению, что подтвердили графики изменения метрик на обучающей и валидационной выборках.

Интеграция моделей в среду Telegram осуществлена через создание бота, который в режиме реального времени принимает входящее сообщение (текст или голос), определяет доминирующую эмоцию и выдаёт поддерживающее сообщение, сформированное на основе лингвистических паттернов эмпатии. Такой подход обеспечивает быстрое эмоциональное облегчение пользователя.

Дополнительно реализован функционал, позволяющий боту формировать отчёты о распределении эмоциональных состояний пользователей за текущий день и текущую неделю. Отчёт за текущий день содержит сводку по количеству зафиксированных эмоций за прошедшие сутки, а за неделю — все записанные эмоции за последние семь дней с автоматическим выявлением самой частой эмоции. При этом бот отправляет рекомендацию, соответствующую наиболее популярному эмоциональному состоянию, что делает отчёты практически применимыми для мониторинга психологического состояния пользователя.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Machová K., Szabóová M., Paralič J., Mičko J. "Detection of Emotion by Text Analysis Using Machine Learning" [Электронный ресурс] // *Frontiers in Psychology*. — 2023. — Vol. 14. — Article 1190326. — Режим доступа: <https://www.frontiersin.org/articles/10.3389/fpsyg.2023.1190326/full> (дата обращения: 16.04.2025).
2. Murthy A. R., Kumar K. M. "A Review of Different Approaches for Detecting Emotion from Text" [Электронный ресурс]. — 2021. — Режим доступа: https://www.researchgate.net/publication/350491911_A_Review_of_Different_Approaches_for_Detecting_Emotion_from_Text (дата обращения: 16.04.2025).
3. Seyeditabari A., Tabari N., Gholizadeh S., Zadrozny W. "Emotion Detection in Text: Focusing on Latent Representation" [Электронный ресурс]. — 2019. — Режим доступа: <https://arxiv.org/abs/1907.09369> (дата обращения: 16.04.2025).
4. Honnappa S. "Speech Emotion Detection Using Voice & Transcribed Text — Part 1" [Электронный ресурс]. — 2020. — Режим доступа: <https://medium.com/analytics-vidhya/speech-emotion-detection-using-voice-transcribed-text-part-1-ce1b9e3e3c12> (дата обращения: 06.04.2025).
5. Malloch Y., Feng B. "What Motivates People to Support?: Impacts of Message Valence and Self-Efficacy on Linguistic Features of Response" [Электронный ресурс] // *Frontiers in Psychology*. — 2022. — Vol. 13. — Article 798205. — Режим доступа: <https://www.frontiersin.org/articles/10.3389/fpsyg.2022.798205/full> (дата обращения: 16.04.2025).
6. Полянский А. И., Быковская Л. И. «Алекситимия у современной молодежи: особенности проявлений у студентов технических и гуманитарных направлений подготовки» [Электронный ресурс]. — 2020. — Режим доступа: <https://cyberleninka.ru/article/n/aleksitimiya-u-sovremennoy-molodezhi-osobennosti-proyavleniy-u-studentov-tehnicheskikh-i-gumanitarnyh-napravleniy-podgotovki> (дата обращения: 11.04.2025).
7. Кудайбергенова С. К., Камзанова А. Т., Жолдасова М. К., Мадалиева З. Б., Несипбаев С. А., Кустубаева А. М. «Исследование взаимосвязи депрессии и личностных особенностей» [Электронный ресурс]. — 2020. — Режим доступа: <https://journals.nauka-nanrk.kz/bulletin-science/article/view/901> (дата обращения: 10.04.2025).

Приложение А Ссылки на исходные коды

Ссылка на репозиторий с исходным кодом проекта

https://github.com/CandyGoose/Neurotechnology_and_efficient_computing